QUANSER
INNOVATE·EDUCATE

Lab Procedure
Brushed DC Motor

Introduction

Ensure the following:

1. You have reviewed the Application Guide – Brushed DC
2. Make sure you have the provided **brushed DC motor**.
3. Make sure the **Mechatronic Actuators Trainer** is out of its case; it is powered using the provided power supply and connected to the PC.
    a. Turn it on using the Power switch on the side of the device.
    b. Both the USB and Power LED should be green.

Driving a Brushed DC Motor

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **File** > **Open Folder**..., and browse to the directory containing the python files and lab procedure for the Brushed DC motor lab (.../actuators_trainer/1_fundamentals/brushed_dc/hardware/python). Open this directory.
2. Open dc_motor_raw.py and locate the following line:

    ```
    with ActuatorsTrainer(block = 2) as actuators:
    ```

    This line is present in all labs and is used to initialize the device. The parameter **`block = 2`** dictates what block, as printed on the device, you are going to use. You can update this parameter if you want to connect the motors to a different block.
3. Connect your motor to the block you will be using.
    a. Use the **`Brushed DC`** connection for the motor power input.
    b. Use the **`Encoder`** connection for the encoder channel.
    NOTE: The `C` slot on the encoder connector is empty since encoders only have `A` and `B` pulses. Make sure you connect it to match the empty connector to the C marking in the board. The `C` slot is there to support brushless DC motors.
4. Connect one of the provided metal fittings to the top of the motor as shown in the picture below. This will help you rotate the shaft easier and observe changes.

5. Run dc_motor_raw.py using the ▷ button or the terminal. Notice that the **`update_dc_individual()`** method near the end of the file has the **`left`** and **`right`** components both set to 0.0. As such, the motor should not turn. Try turning the motor by hand while the script is running, and again after the script completes executing. Note any differences. Note that the **`left`** and **`right`** parameters are mapped to the A and B channels respectively, as marked in the device for the Brushed DC motor connector.

6. Stop the script using **`Ctrl+C`** (the script stops after 5 seconds automatically). Run the script again multiple times updating the **`left`** value to 0.1, 0.2, etc. up until 1.0, while the **`right`** parameter is still set to 0.0. These 0.0 to 1.0 values are mapped to $0 - 12V$ for the motor. What do you observe? Note the direction the motor turns in.

7. Repeat step 6 with the **`right`** value set to 0.1, 0.2, etc. up until 1.0, with the **`left`** value set to 0.0. What do you observe?

8. Try repeating step 6 with negative values for either **`left`** OR **`right`** between -1.0 and 0.0. Does the motor move?

9. Repeat step 6 with the **`left`** AND **`right`** parameters set to a variety of positive values between 0.0 and 1.0. What do you observe?

10. Remove the metal fitting you added earlier on.

Characterizing a Brushed DC Motor

11. Open the dc_motor_drive.py file.

    a. Update the initialization line with the block you will be using.

    b. Connect the DC motor and the encoder to the device as described in the first 3 steps of the Driving a Brushed DC Motor section of this lab.

    c. This file will apply a square wave to the motor, so it might feel like it jumps every time the change happens. It is normal.

12. Note that the **`update_dc_individual()`** function is no longer used near the end of this file. Rather, **`update_dc()`** is used, which automatically writes to the left or right channel based on the command. Both of these functions do not take voltage directly, but instead a PWM value from 0 to 1 that will be mapped from 0 to 12 V.
    NOTE: You can right-click the **`update_dc()`** method and go to definition, which should show you the logic to enable bi-directional control of the motor.

13. Run the script either using the ▷ button or the terminal. The connected motor should rotate, and four Python Scopes should pop up on your desktop. Close the scope labeled **Motor Model**, which will be used later in the lab. What do these scopes show? What relationship can you draw between the data shown on the three scopes? Re-run the script if required.

14. Change the **`squareWaveVoltage`** near the top of the file from 12V down to 6 Volts and repeat. What do you observe?

15. Set the **squareWaveVoltage** back to 12 Volts. Locate the line near the end of the file **actuators.update_dc(dc_command/12, limitCmd=False)** and run the script while the **limitCmd** option is set to **False**, set it to **True** and run again. What do you observe? When finished, set **limitCmd** back to **False** and continue.

Sensing

16. Re-run dc_motor_drive.py.
    a. Observe the encoder and tachometer scopes, what are their units?
    b. Stop the script by using **Ctrl+C** and change the encoderRatio in the experiment constants section of the script from 1 to $1/4096$. Look at the DC motor encoder datasheet and observe how many points per revolution are given by the device. Using quadrature decoding, as described in the encoder concept review, the number of points gets multiplied by x4. What does this change do to the encoder and tachometer data? What are the units of the data at this point?
17. Repeat the previous step with the encoderRatio set to $2\pi/4096$. What are the units of the encoder and speed data now?
    Hint: You can use π by using **np.pi** in the script.
18. Open the datasheet for the brushed DC motor you are using located in the brushed_dc folder and find the gear ratio/reduction ratio parameter. If using the provided brushed DC motor, you should see a value of 27. Adjust the **encoderRatio** gain considering the reduction ratio to properly measure the output shaft speed. What does this do? What are the units of the data now? Verify visually if you can.

Modeling

19. Vary the amplitude using **squareWaveVoltage**, from 2/4/6/8/10/12 volts.
20. Run the script for each value and record the observed speed. From the scope, complete the table 1.

Voltage (V)	Speed (rad/s)
2	
4	
6	
8	
10	
12	

Table 1: range of speeds at various voltages

21. Using either excel or your preferred tool, analyze the relationship between the applied voltage and the observed speed. Is this relationship linear? Estimate the linear gain constant that relates the voltage and speed.

22. Set the **`squareWaveVoltage`** back to 12. Set the **`speedGain`** under the **`# model parameters`** section to the value you found from the previous step.

23. Run the script and focus on the Motor Model scope this time. Does the modeled speed and measured speed track? Adjust the **`speedGain`** if they do not.

24. In the same section where **`speedGain`** was modified, vary the **`beta`** parameter from 0.05 to 0.25 in increments of 0.05. What do you observe? Set it to a value that makes the modeled and measured speeds match the best. This parameter is used in a discrete first order delay (**`mdlSpeed`**). How does this parameter relate to the time constant of the system?

25. Stop, save and close your program.

26. Power OFF the Mechatronic Actuators Trainer, disconnect any motors and pack them along the power supply and USB cables in the provided case.